CHAPTER 1

PREAMBLE

The preamble comprises of problem statement, solution, objectives, literature review and organization of the report.

1.1 PROBLEM STATEMENT

Academic and professional certificate fraud has become a prevalent issue in today's digital world. Traditional verification processes often rely on manual checks, centralized databases, or email-based verifications, all of which are prone to forgery, manipulation, and human error. These methods are time-consuming, inefficient, and lack transparency, making it challenging for employers, academic institutions, and other entities to validate credentials accurately. With the increasing number of graduates and certifications being issued each year, the pressure on existing systems grows exponentially, further amplifying their limitations.

Current centralized systems are vulnerable to data breaches and hacking attempts, risking the authenticity of sensitive information. Third-party validation services, though helpful, are costly and rely on trust in the intermediaries, which can lead to dependency and security risks. Additionally, these systems fail to provide a tamper-proof and transparent way to track certificate authenticity, leaving loopholes for fraudsters to exploit.

The lack of a robust and universally trusted mechanism for certificate management and verification creates challenges in sectors such as education, employment, and professional development. There is a growing need for a solution that not only validates certificates but also ensures their security, transparency, and immutability.

1.2 THE SOLUTION

Our solution addresses as

The Blockchain-Based Certificate Authentication System leverages blockchain technology to address the issues of certificate fraud, mismanagement, and inefficiencies. Blockchain, with its decentralized, immutable, and transparent nature, serves as the backbone of the proposed system. By utilizing the Ethereum blockchain and integrating it with off-chain storage solutions, the system provides a secure, tamper-proof, and transparent platform for issuing, storing, and verifying academic and professional certificates.

This system ensures that certificates are digitally signed using cryptographic techniques, making them nearly impossible to forge. Each certificate is assigned a unique hash stored on the blockchain, which acts as a permanent record of authenticity. The use of smart contracts automates the verification process, ensuring efficiency and eliminating human error. Institutions can generate certificates directly on the platform, while employers and other stakeholders can validate them using the certificate's unique ID.

Incorporating tools such as Ganache, Truffle, and MetaMask simplifies blockchain interactions, and SendGrid is used for sending certificates securely via email. The system also employs MongoDB for off-chain data storage, ensuring scalability to handle large volumes of certificates. Hosted on platforms like Heroku, it ensures availability and seamless user experience.

This solution not only eliminates certificate fraud but also fosters trust among academic institutions, employers, and individuals, creating a more efficient and reliable credentialing ecosystem.

1.3 OBJECTIVES OF PROJECT

The primary objective of the Blockchain-Based Certificate Authentication System is to provide a secure, efficient, and reliable method for issuing and verifying educational certificates using blockchain technology. The specific objectives of the project include:

1. **User Authentication:** To enable secure user registration and login functionalities, ensuring that only authorized users can access the system. This includes implementing validation checks to safeguard user information and improve overall system security.
2. **Certificate Issuance:** To automate the process of issuing certificates upon successful completion of a program. The system will generate unique certificate IDs and send them to users via email, allowing for easy retrieval and verification.
3. **Data Integrity and Security:** To ensure the integrity of certificate data by storing transaction records in a secure database. This involves hashing user passwords and safeguarding sensitive information to prevent unauthorized access.
4. **Email Notification System:** To implement an efficient email notification system that promptly informs users of their issued certificates. This feature aims to enhance user engagement and facilitate easy access to certificate details.

5. **Certificate Verification:** To provide an accessible method for users and third parties to verify the authenticity of certificates. This is crucial for maintaining trust in the educational credentials presented by individuals.
6. **Admin Functionality:** To empower administrators with the ability to manage user data and oversee the certificate issuance process. This includes updating user information and addressing any issues that arise.

1.4 ADVANTAGES

The adoption of blockchain technology introduces significant improvements in certificate management. Key advantages include:

1. **Security**: The decentralized and cryptographically secured nature of blockchain ensures that records are tamper-proof and resistant to hacking or unauthorized access.
2. **Transparency**: Blockchain's public ledger allows verifiers to access certificate information in real time, eliminating trust issues.
3. **Efficiency**: Automation through smart contracts speeds up the validation process, saving time and resources.
4. **Cost-Effective**: By removing intermediaries and manual processes, the system reduces costs associated with certificate issuance and verification.
5. **Scalability**: Blockchain technology can support large volumes of certificates and transactions, making it suitable for institutions of any size.
6. **Global Applicability**: The system can be accessed and used worldwide, facilitating international certificate validation.

1.5 LITERATURE SURVEY

[1]. Blockchain was introduced long before Bitcoin was introduced by Satoshi Na-kamoto. However, blockchain gained its popularity with Bitcoin. Al though bitcoin is sometimes referred to as a blockchain by some people, but the fact is bitcoin was built using blockchain technology. In a short period of time, blockchain became popular when people discovered its salient benefits. It has become one of the top technologies worldwide. For this reason, many studies and research were conducted and this proposed system rewired some of them.

[2]. Authors of have made some excellent points about blockchain architecture and consensus algorithms. They have also examined the limitations of blockchain and given some possible solutions. They also investigate various consensus algorithms in different respects and discuss their differences. By analyzing those differences, they can determine the most appropriate consensus algorithm.

[3]. The author of Rishabh Garg, has examined how blockchain technology can be applied to solve problems within academic institutes and the employment sector. According to the author, blockchain will be the panacea for this issue. He proposed a framework that can issue new certificates as well as vali date them.

[4]. In Osman Ghazali and Omar S. Sale, emphasized the significance of certificate verification and its influence in our society. In the paper, traditional verification processes were briefly explained, as well as their limitations. To eliminate those limitations, they proposed a blockchain-based verification system. A system that is not only used for verification but also for generating new certificates. They will generate digitally signed certificates using the asymmetric key and timestamp. Students will receive a copy, which they can verify in the system.

[5]. Jayesh G. Dongre and his colleagues proposed to solve the problems of the current system of certificate verification. In the paper, they talked about the current verification process and the proliferation of certificate fraud. Using blockchain, they have developed a platform for validating and generating certificates. In their view, the use of blockchain for certificate verification is beneficial to society. It will eliminate certificate fraud.

[6]. R. Suganthalakshmi and her colleagues investigated an optimal method of verifying academic credentials in 2022. The number of graduating students is increasing every year, and it is becoming increasingly necessary to validate their academic credentials. Without proper validation, any ineligible candidate will be blessed with opportunities. Their solution is to create a platform for all certificates a student may possess. Students upload all their credentials to the system, and the system stores them on the blockchain. To verify a credential, a person needs the student's ID and password. They used the PoW (Proof of work) consensus algorithm for validation, but the validation method was not well elucidated.

In the papers mentioned, researchers are interested in building a system to verify and generate academic credentials. Although their system design is different from each other but they all agree on using blockchain for security. Our proposed system can also generate and verify academic credentials, as well as modify them. There is a possibility of making a mistake when generating certificates. Our study found that none of the papers offered editing options if any modifications were needed. We provide an editing facility option to fix those errors.

1.6 ORGANIZATION OF REPORT

Chapter 1 Preamble:

The preamble in a chapter provides a concise introduction, outlining the chapter's main themes and objectives. It may include background information and context to help readers understand the subject matter. Additionally, the preamble serves as a roadmap, guiding readers through the upcoming content**.**

This section includes:

1. Problem statement
2. The solution
3. Objectives of the project
4. Advantages of the project

Chapter 2 Software Requirement Specification:

A Software Requirements Specification (SRS) is a concise document.
It outlines the functional and non-functional requirements of a software system, providing a blueprint for development and ensuring alignment on project goals and constraints.

This section includes:

1. Functional overview
2. User characteristics
3. Input Requirements
4. Output requirements
5. Functional requirements
6. Software Requirements

7. Hardware requirements
8. Project cycle

Chapter 3 System Design:

System design involves the process of creating a detailed blueprint for a software system, including its architecture, components, interfaces, and interactions. It encompasses defining system requirements, designing data structures, algorithms, and interfaces, and making decisions about technologies, platforms, and frameworks to be used.

This section includes:

1. Project Architecture
2. Database design
3. Sequence diagram
4. Dataflow diagram
5. Database table structure
6. Flow chart
7. Utilities

Chapter 4 Implementation:

Implementation refers to the process of translating a design or plan into a working software system. It involves writing code, integrating components, and testing the system to ensure that it functions according to the specified requirements.

Implementation typically follows the system design phase and involves programming, debugging, and optimizing the software to meet performance, reliability, and usability goals. Successful implementation results in a fully functional software product ready for deployment and use by end-users.

This section includes:

1. Software Tools used
2. Implementation details
3. Front end forms deign

Chapter 5 Testing:

Testing is the process of evaluating a software system to identify defects, errors, or discrepancies between expected and actual outcomes. It involves executing test cases, scripts, or scenarios to verify that the software functions correctly and meets specified requirements. Testing encompasses various techniques such as unit testing, integration testing, system testing, and acceptance testing, aiming to ensure the quality, reliability, and robustness of the software.

This section includes:

1. Scope
2. Unit Testing
3. Integration Testing
4. Functional Testing
5. System Testing

Chapter 6 Results:

The results section summarizes and presents the findings of the study.

This section includes:

1. Snapshot of projects

Chapter 7 Conclusion:

The conclusion section summarizes the main findings and outcomes of the project, offering insights into its achievements, challenges, and potential future directions. It provides closure to the report and highlights the project's contributions to the field of software development.

This section includes:

1. Conclusion
2. Future Scope

Appendix:

The appendix section of a document typically contains supplementary materials that support or enhance the main content. These materials may include additional data, charts, graphs,

tables, code samples, or other relevant information that provides further context or detail. The appendix serves as a reference for readers who may want to delve deeper into specific aspects of the document or need additional information to better understand the content.

References:

The references section of a document lists all the sources cited or consulted during the research or writing process. It provides readers with the necessary information to locate and verify the credibility of the sources used in the document. References are typically listed alphabetically by the author's last name or by the title of the source if no author is available. Each reference includes essential details such as the author's name, publication year, title of the work, publisher, and relevant page numbers or URLs.

CHAPTER 2

SOFTWARE REQUIREMENT SPECIFICATION

A Software Requirements Specification (SRS) is a comprehensive document detailing the functional and non-functional requirements of a software system, serving as a blueprint for development.

2.1 FUNCTIONAL OVERVIEW

The system is designed to streamline the process of certificate issuance, storage, verification, and management using blockchain technology. Its key functionalities include:

1. **Certificate Issuance:**
   Institutions can issue digitally signed certificates through an intuitive interface. The system allows for uploading certificate details, including recipient information and achievements.
2. **Blockchain Storage:**
   Each certificate is converted into a unique hash record stored on the Ethereum blockchain. The immutability of blockchain technology ensures that these records cannot be altered or forged.
3. **Certificate Management:**
   Includes functionalities to search and retrieve specific certificates from the database. If modification is enabled, institutions can update certificate details. Tracking features help institutions monitor the issuance and status of certificates.
4. **Certificate Verification:**
   Verification can be done by scanning a QR code embedded in the certificate or entering the certificate's unique ID. The system validates this information against blockchain data, providing instant and reliable verification.
5. **Off-chain Storage:**
   MongoDB is utilized for storing additional metadata that doesn't need to be on the blockchain. This setup allows for greater scalability and efficient data management.

6. **Secure Distribution**:

   SendGrid is integrated to securely distribute certificates via email. Ensures that recipients receive their certificates promptly and securely.

7. **Blockchain Development Tools:**

   Ganache provides a personal blockchain for Ethereum development, allowing for the creation and testing of smart contracts. Truffle is a development framework that simplifies the process of developing and deploying smart contracts.

8. **Wallet Interface**:

   MetaMask is used as the wallet interface, facilitating Ethereum transactions.
   Provides a secure and user-friendly way to interact with the blockchain.

9. **Responsive Front End:**

   The system is designed to be responsive, ensuring accessibility across various devices such as desktops, tablets, and smartphones. Enhances user experience by providing seamless access and interaction.

2.2 USER CHARACTERISTICS

The platform is tailored to meet the needs of various user groups, ensuring ease of use and accessibility. The primary users include:

1. **Academic Institutions:**

   Administrators create and manage certificates. They require minimal blockchain knowledge, as the interface abstracts technical complexities.

2. **Employers/Verifiers:**

   Users verify certificates by scanning QR codes or entering unique IDs. They benefit from instant, reliable, and tamper-proof verification.

3. **Students/Certificate Holders:**

   Users receive digitally signed certificates and can share them with employers or institutions. They use the platform primarily for retrieval and sharing purposes.

4. **Admin Onboarding:**

   Admins receive comprehensive training sessions to manage certificate creation, updates, and tracking. Detailed manuals and guides are provided to ensure they understand all functionalities and best practices.

5. **Basic Digital Literacy:**

   Users should know how to use web browsers and navigate through the platform's interface.

2.3 INPUT REQUIREMENTS

Inputs include certificate details (name, date, credentials), blockchain wallet keys and unique IDs for verification purposes.

2.3.1 SERVER-SIDE REQUIREMENTS

The server side is the backbone of the system, handling blockchain transactions, certificate management, and user authentication. Key requirements include:

1. **Operating System**

   A Linux-based OS like Ubuntu is recommended for its reliability, security, and performance. Ubuntu is widely used for server environments due to its stability and vast community support.

2. **Programming Environment**

   Node.js for back-end development. It is chosen for its asynchronous nature and ability to handle a high volume of requests efficiently. It is widely used in blockchain and scalable server applications.

3. **Database**

   MongoDB - A NoSQL database for storing off-chain data, such as certificate metadata, user details, and transaction logs. Its scalability and flexibility make it ideal for managing dynamic datasets.

4. **Blockchain Tools**

   Ganache is a local Ethereum blockchain emulator used for testing and simulating blockchain transactions during development. Truffle is a framework for developing, deploying, and testing Ethereum smart contracts. It simplifies blockchain development with tools like migrations and contract abstractions.

5. **APIs and Libraries**

   Web3.js is a JavaScript library for interacting with the Ethereum blockchain. It enables wallet transactions, smart contract calls, and data retrieval. SendGrid is an email API for sending transactional emails, such as certificate issue notifications or alerts.

6. **Hosting Platform**

   A cloud platform for deploying and maintaining the server-side application. It supports seamless scaling and provides tools for continuous integration and monitoring.

2.3.2 CLIENT-SIDE REQUIREMENTS

The client-side ensures user interaction through an intuitive interface. Key requirements include:

1. **Operating System**

   The interface should work seamlessly across major OS platforms, including Windows, macOS, and Linux, ensuring broad compatibility.

2. **Browser**

   Modern web browsers such as Chrome, Firefox, or Edge are necessary. They also require the MetaMask extension, a browser-based wallet for interacting with blockchain networks.

3. **Frameworks**

   React.js a JavaScript framework for creating dynamic, responsive, and interactive user interfaces. Its component-based architecture ensures reusability and efficient rendering.

4. **Libraries**

   Axios is a promise-based HTTP client for making API requests to the server, such as fetching certificate details or submitting verification requests.

2.4 OUTPUT REQUIREMENTS

The outputs include verified certificate statuses and error notifications.

2.4.1 CRITICAL INFORMATION

Critical Information includes:

1. **Certificate Validity and Authenticity:** The system verifies and outputs the validity and authenticity of certificates.
2. **Issuer Details and Timestamps**: Displays details of the certificate issuer and associated timestamps for verification.

3. **Verification Process:** Verifiers use unique IDs or QR codes to confirm certificate authenticity against blockchain data.
4. **Notifications for Invalid Certificates:** The system provides clear notifications for expired or invalid certificates, ensuring accurate information.

2.4.2 ERROR MESSAGES

In case of errors or issues encountered during data retrieval and comparison, the system is going to provide clear and concise error messages. These error messages will:

1. **Certificate not found**: This error message appears when the entered certificate ID does not match any records in the system.
2. **Network error:** This error occurs when there are connectivity issues that prevent the system from accessing the necessary data.
3. **Unauthorized access:** This message is displayed when a user attempts to access the system with invalid credentials.

2.5 FUNCTIONAL REQUIREMENTS

Functional requirements outline the specific functionalities or features that a software system must possess to fulfill its intended purpose and meet the needs of its users. These requirements describe what the system should do, including its capabilities, behaviours, and interactions with users and other system components.

1. **Certificate Creation**

   Admins can generate certificates by entering user details. Certificates are hashed and stored on the Ethereum blockchain.

2. **Certificate Verification**

   Verifiers use a QR code or ID to check certificate authenticity. The system retrieves blockchain data to confirm validity.

3. **User Authentication**

   Role-based access ensures secure operations for admins and students. Blockchain wallets are used for admin authentication.

4. **Scalability**

   MongoDB is integrated for off-chain data, allowing efficient storage for millions of certificates.

5. **Integration with Blockchain**

   Smart contracts automate certificate management, ensuring secure transactions.

6. **Notification System**

   Certificates are sent to users through email via SendGrid.

   Notifications for certificate issuance or verification completion are included.

2.6 SOFTWARE REQUIREMENTS

Table 2.1 represents the software requirements.

Table 2.1 software requirements

Software	Requirement
Operating system	Windows 7 or more
Blockchain Framework	Ethereum, Truffle, and Ganache
Programming Languages	JavaScript and Solidity
Libraries/APIs	Web3.js, Axios, QR Code libraries, and SendGrid API.
Database	MongoDB

2.7 HARDWARE REQUIREMENTS

Table 2.2 represents the hardware requirements.

Table 2.2 Hardware requirements

Hardware	Requirement
RAM	8 GB or more
Processor Speed	1.5 GHz or more
Hard Disk	128 GB or more
Processor	Intel core i3 or more

2.8 PROJECT CYCLE

The project cycle, also known as the software development life cycle (SDLC), refers to the stages involved in the development of a software project from initiation to completion.

Waterfall is a linear approach to software development, where each phase must be completed before moving on to the next. It's often pictured as a cascading waterfall, where each step feeds into the next.

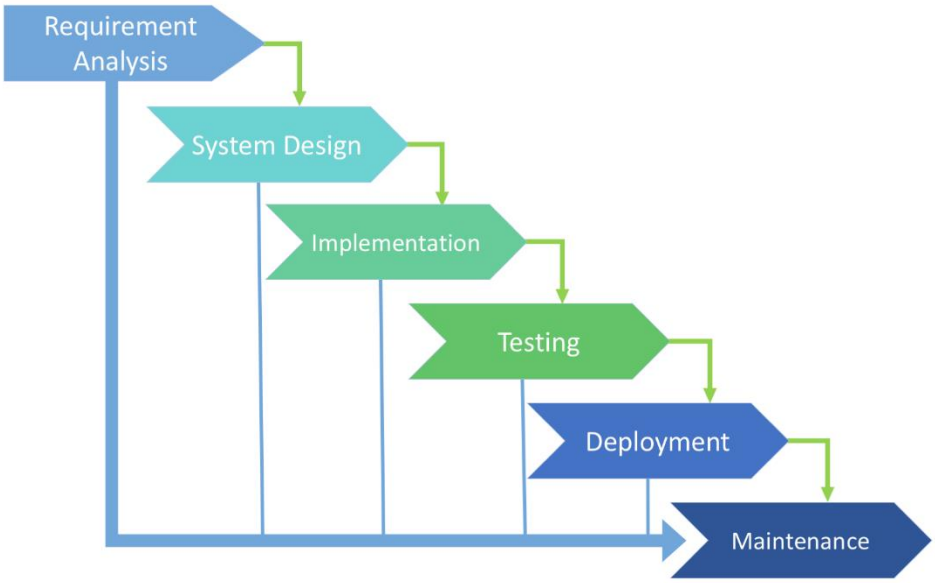


Figure 2.1 Project Cycle

Figure 2.1 represents project cycle which comprises of the following

- **Requirements Analysis:**
  Identifying requirements, stakeholders, and tools. Establishing a clear roadmap and dividing tasks into milestones.
- **System Design**:
  Drafting system architecture, creating data flow diagrams, and finalizing the user interface layout. Smart contract design is prioritized for secure blockchain interactions.
- **Implementation:**

  Implementing the back-end using Node.js, developing smart contracts with Solidity, and integrating MongoDB for off-chain storage. React.js is used for client-side development

- **Testing:**

  Conducting unit, integration, and system testing to ensure seamless interactions among components. Tools like Truffle and MetaMask are used to test smart contracts.

- **Deployment:**

  Deploying the application on Heroku and connecting it to the Ropsten Test Network for blockchain functionality. MetaMask wallets are configured for real-time use.

- **Maintenance:**

  Providing ongoing support and updates to ensure the system remains secure and functional. Monitoring system performance, addressing bugs, and implementing new features as needed. Regularly updating the smart contracts and underlying technologies to adapt to evolving requirements and advancements in blockchain technology.

CHAPTER 3

SYSTEM DESIGN

This chapter includes project architecture and description, database design, sequence diagrams, data flow diagrams, database table structures, flowchart and utility.

3.1 PROJECT ARCHITECTURE AND DESCRIPTION

The project architecture section typically provides an overview of how the different components and modules of a system are structured and interact with each other. It outlines the high-level design, including the technologies used, data flow, and the relationships between various elements.

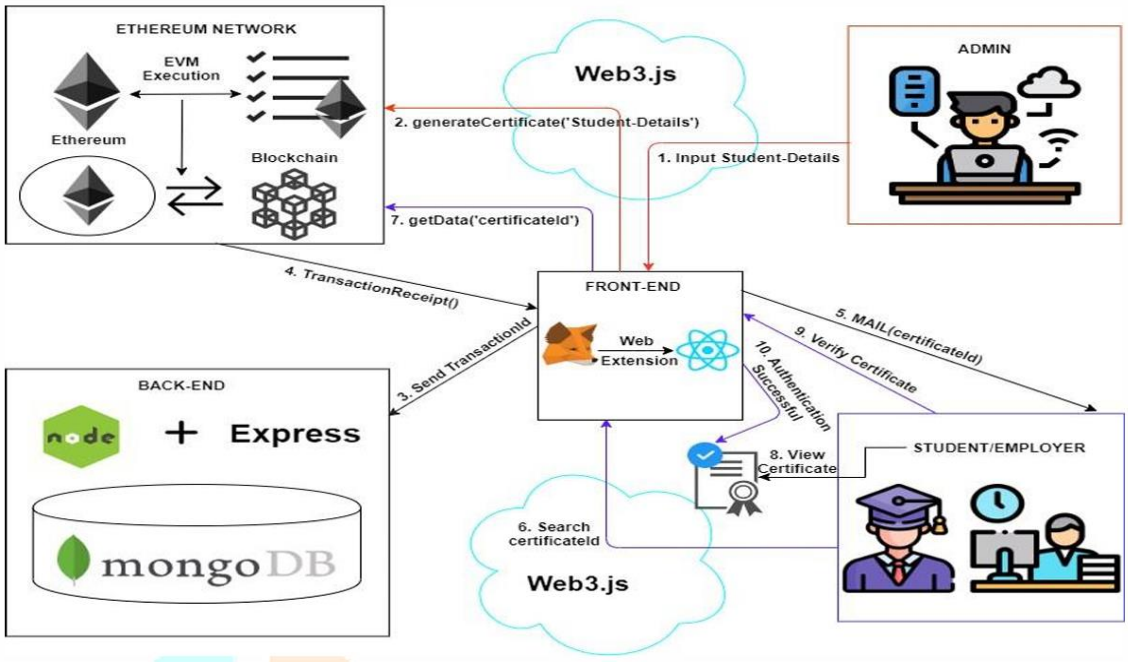


Figure 3.1 Project Architecture

Figure 3.1 represents the project architecture. The diagram represents a blockchain-based system for certificate issuance and verification, leveraging decentralized and centralized technologies to ensure security and transparency. The process begins with an admin who inputs student details through a frontend application. This frontend uses Web3.js, a JavaScript library for interacting with the Ethereum blockchain, along with a web extension like MetaMask for managing transactions.

The admin submits certificate details, which are sent to the backend built with Node.js and Express. The backend facilitates communication between the frontend and the blockchain while also interacting with a MongoDB database. The database stores additional metadata, such as user contact information, which complements the data stored on the blockchain. The blockchain receives the hashed certificate data, which is immutable and secure, ensuring authenticity.

Once the transaction is confirmed, a receipt is returned to the frontend. The certificate, along with verification details, is then emailed to the recipient. Students or employers can later search for certificates using unique IDs through the system. This triggers a process where the hash stored on the blockchain is retrieved and compared with the submitted details to validate authenticity.

The verification ensures data integrity, as blockchain technology prevents tampering. Users can view or validate certificates via the frontend, with the process being both decentralized and reliable. Combining blockchain's immutability with the flexibility of centralized storage for metadata offers a robust solution for secure certificate issuance and validation. This architecture ensures trust, transparency, and convenience for all stakeholders involved.

3.2 DATABASE DESIGN

The database diagram illustrates the integration of MongoDB and blockchain technology for secure academic certificate verification. It depicts how certificate data flows between the database and the blockchain, ensuring data integrity and transparency

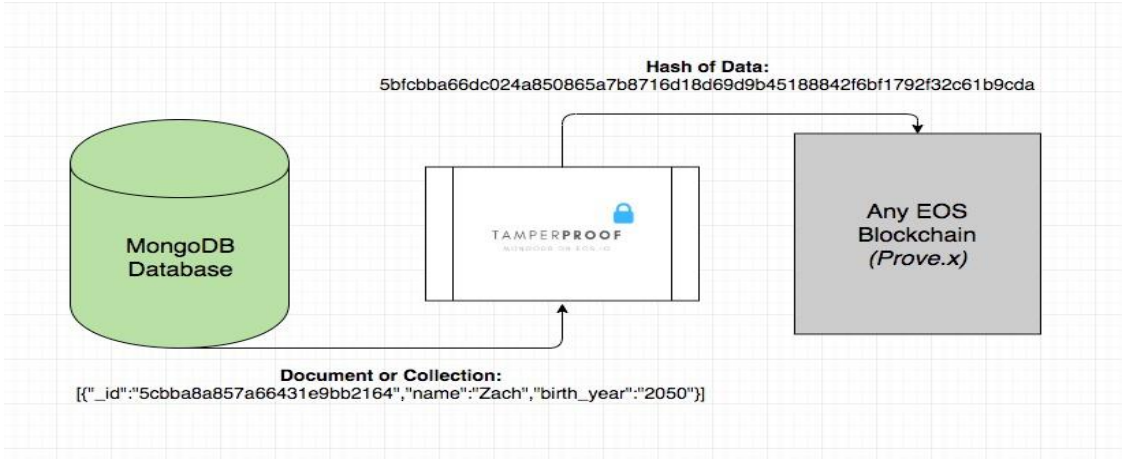


Figure 3.2 Database design

Figure 3.2 gives MongoDB acts as the primary storage for certificate metadata. This metadata includes details like student names, certificate types, issuance dates, and unique identifiers. When a certificate is issued, the certificate's metadata or content is hashed using a cryptographic hashing algorithm. The hash uniquely represents the certificate data without revealing the actual content, ensuring data privacy.

The generated hash is then sent to the blockchain, where it is securely stored. By recording the hash on the blockchain, the system leverages blockchain's immutable and decentralized nature to create a tamper-proof record. The blockchain acts as a public ledger, allowing third parties to verify the authenticity of the certificate without accessing its sensitive details.

During the verification process, the verifier inputs a certificate ID or scans a QR code. The system retrieves the certificate metadata from MongoDB and the corresponding hash from the blockchain. The metadata is rehashed, and the newly computed hash is compared with the one stored on the blockchain. If the hashes match, it confirms that the certificate is authentic and unaltered. The system then communicates the result to the verifier.

This approach ensures that the academic certificates are secure, tamper-proof, and easily verifiable, combining MongoDB's efficient storage with blockchain's robust security and transparency.

Key Features of sqlite3 are:

1. **Tamper-Proof Records:** Blockchain ensures certificate data integrity by storing cryptographic hashes that cannot be altered.
2. **Data Privacy:** Sensitive metadata remains securely in MongoDB, with only the hash stored on the blockchain.
3. **Real-Time Verification:** Verifiers can instantly validate certificates by comparing hashes with blockchain records.
4. **Scalable Architecture:** MongoDB supports efficient scaling for handling large volumes of certificates.
5. **Decentralized Validation**: Blockchain allows independent verification of certificates without relying on a central authority.

3.3 SEQUENCE DIAGRAM

A sequence diagram illustrates how objects interact in a particular scenario of a system. It shows the sequence of messages exchanged between objects over time, depicting the flow of control and collaboration among various components.

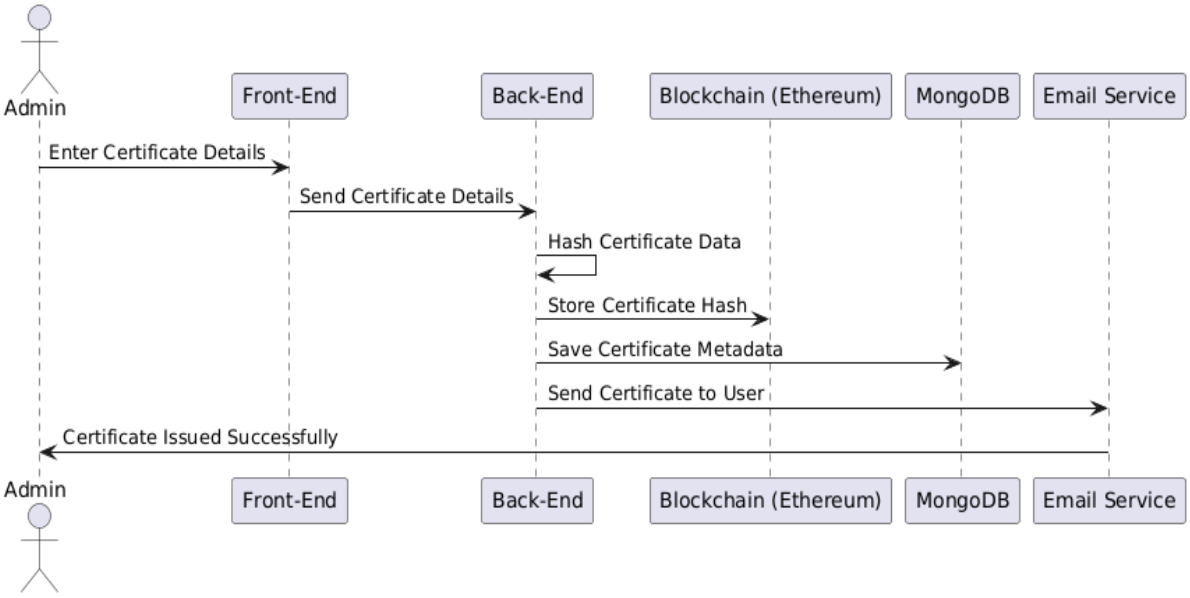


Figure 3.3 Sequence Diagram-Admin

Figure 3.3 represents the sequence diagram of the flow of interactions in the certificate issuance process using blockchain technology. The process begins when the admin inputs the certificate details through the front-end interface. These details are then transmitted to the back-end system, where they are processed. The back-end hashes the certificate data to ensure its integrity and generates a unique identifier. This hashed data is sent to the blockchain, where it is securely stored in a decentralized manner.

The system then stores associated metadata, such as the certificate hash and issuance details, in MongoDB. This enables efficient management and quick retrieval without the need for constant blockchain interaction. Once the storage process is complete, the back-end system uses an email service to send the issued certificate to the intended recipient. The email contains the certificate and its unique identifier, enabling verification later.

After completing these steps, the admin receives confirmation that the certificate has been successfully issued. This process ensures that the certificate is securely stored, efficiently managed, and easily verifiable, leveraging the combined advantages of blockchain, database management, and automated email communication.

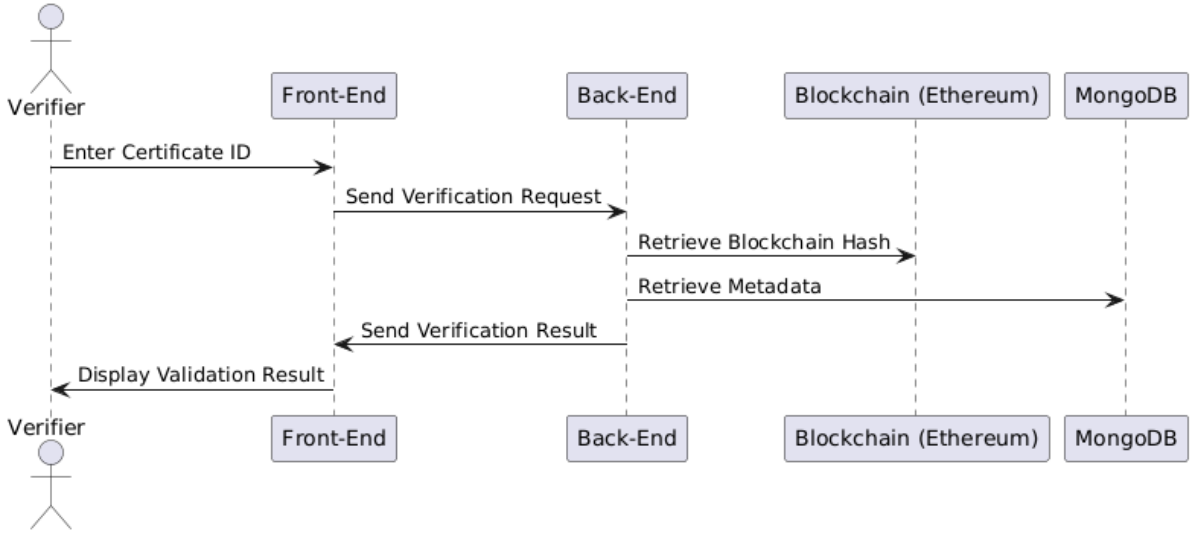


Figure 3.4 Sequence Diagram-Verifier

Figure 3.4 represents the sequence diagram the verification process for certificates using blockchain technology. The verifier begins the process by entering a certificate ID through the front-end interface. This action sends a verification request to the back-end system. The back-end then interacts with both the blockchain and the MongoDB database to validate the certificate.

From the blockchain, the back-end retrieves the hash of the certificate data, which was securely stored during issuance. Simultaneously, metadata associated with the certificate is fetched from MongoDB. The system compares the retrieved data to ensure consistency and authenticity. If the hash and metadata match, the certificate is validated, and the result is sent back to the front-end.

Finally, the front-end displays the validation result to the verifier, confirming the authenticity or invalidity of the certificate. This process ensures a secure, efficient, and transparent verification mechanism, leveraging the decentralized nature of blockchain and the structured storage of MongoDB.

3.4 DATA FLOW DIAGRAM

A data flow diagram (DFD) is a graphic representation of the “flow” of data through an information system. A data flow diagram can also be used for the visualization of data processing (structured design).A first-level Data Flow Diagram (DFD), also known as a Level 0 DFD, provides a broad overview of the entire system and its interactions with

external entities. It represents the highest level of abstraction and decomposes the system into its major processes or subsystems, showing how data flows between them.

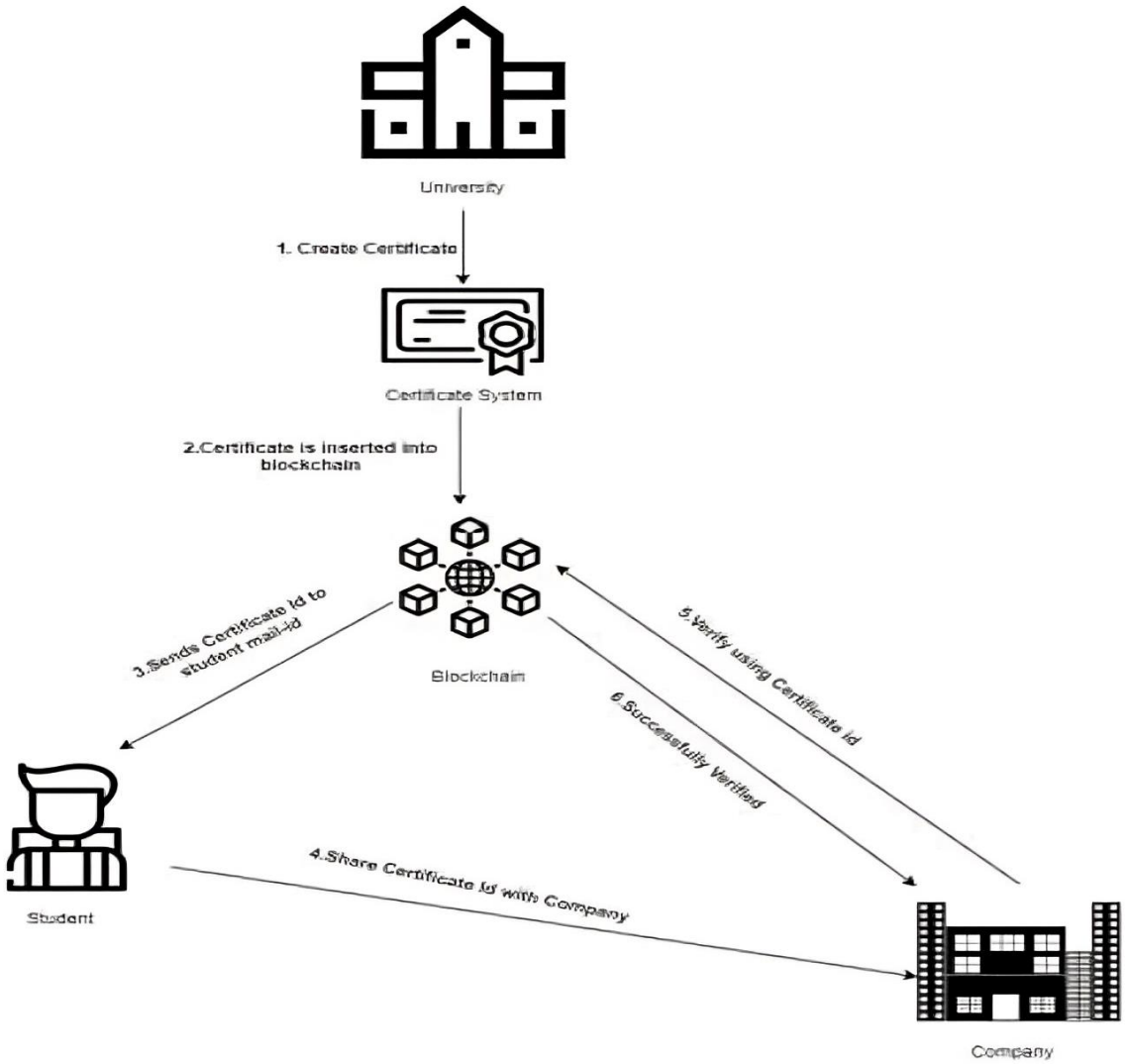


Figure 3.4 Data flow diagram

The figure 3.4 represents the data flow diagram of the Academic certificate verification. The diagram highlights the secure and transparent process of issuing, storing, and verifying certificates using blockchain technology, benefiting universities, students, and companies alike.

In a Level 0 DFD:

1. **Create Certificate:** The university generates a digital certificate for the student using a certificate system.

2. **Certificate Insertion into Blockchain:** The certificate is securely inserted into the blockchain. Blockchain ensures that the certificate data is immutable and tamper-proof.

3. **Sending Certificate ID to Student:** The system sends the unique Certificate ID to the student via their registered email ID. This ID acts as the reference key for retrieving and verifying the certificate.

4. **Sharing Certificate ID with Company:** The student shares their Certificate ID with the company (e.g., during a job application).

5. **Verification Using Certificate ID:** The company uses the Certificate ID to query the blockchain and verify the certificate's authenticity.

6. **Successfully Verified:** If the Certificate ID matches the records stored on the blockchain, the certificate is verified successfully. This assures the company of the certificate's validity.

3.5 DATABASE TABLE STRUCTURE

To understand table structure, we use UML use-case diagram, inorder to first understand the interaction between user and system and then necessary tables required.

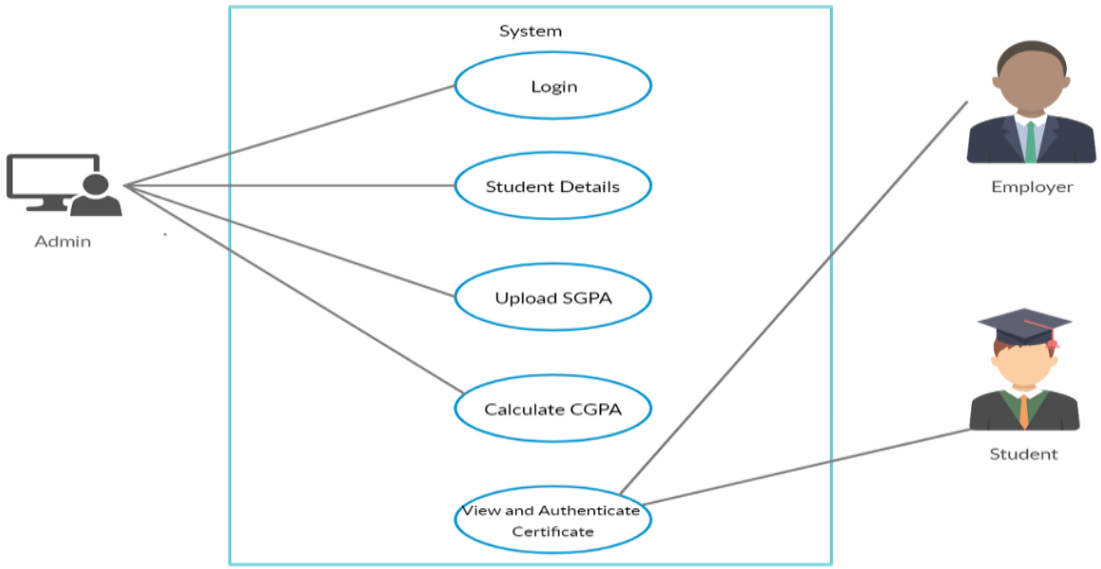


Figure 3.5 Use case diagram

Figure 3.5 shows use case diagram of Academic verification. It helps to understand the system's behaviour from a user's perspective and serves as a blueprint for designing the system's functionality.

Here are key aspects of a use case diagram:

1. **Actors**: In this system, the Admin or University is the primary actor responsible for logging in, generating certificates, and managing the overall system. The Student, who receives the certificate via email, uses the provided Certificate ID to view it. Finally, the Employer verifies the authenticity of the certificate by using the Certificate ID provided by the student.

2. **Use Cases**: The admin logs into the system and accesses various functionalities, such as generating certificates based on student performance data and sending these certificates via email. Each certificate includes a unique Certificate ID for verification purposes. Students can use this ID to view their certificates, and employers can verify the authenticity of the certificates using the same ID, ensuring that they are valid and issued by the university.

3. **Relationships**: The Admin/University manages the system, generates certificates, and sends them via email. The student views the certificate using a Certificate ID, and the Employer verifies the certificate's authenticity with the same ID. Association links actors to use cases, and generalization shows how actors or use cases can inherit properties from others.

4. **System Boundary**: The System Boundary is depicted by a box enclosing the use cases. It defines the scope of the Blockchain-Based Academic Certificate Authentication System, representing the system's functionalities and the interactions of the actors with the system. The boundary shows which actions are handled within the system and which are external.

3.6 FLOW CHART

A flowchart is a graphical representation of a process, algorithm, or workflow. It uses various shapes and symbols connected by arrows to illustrate the steps or actions involved in completing a task.

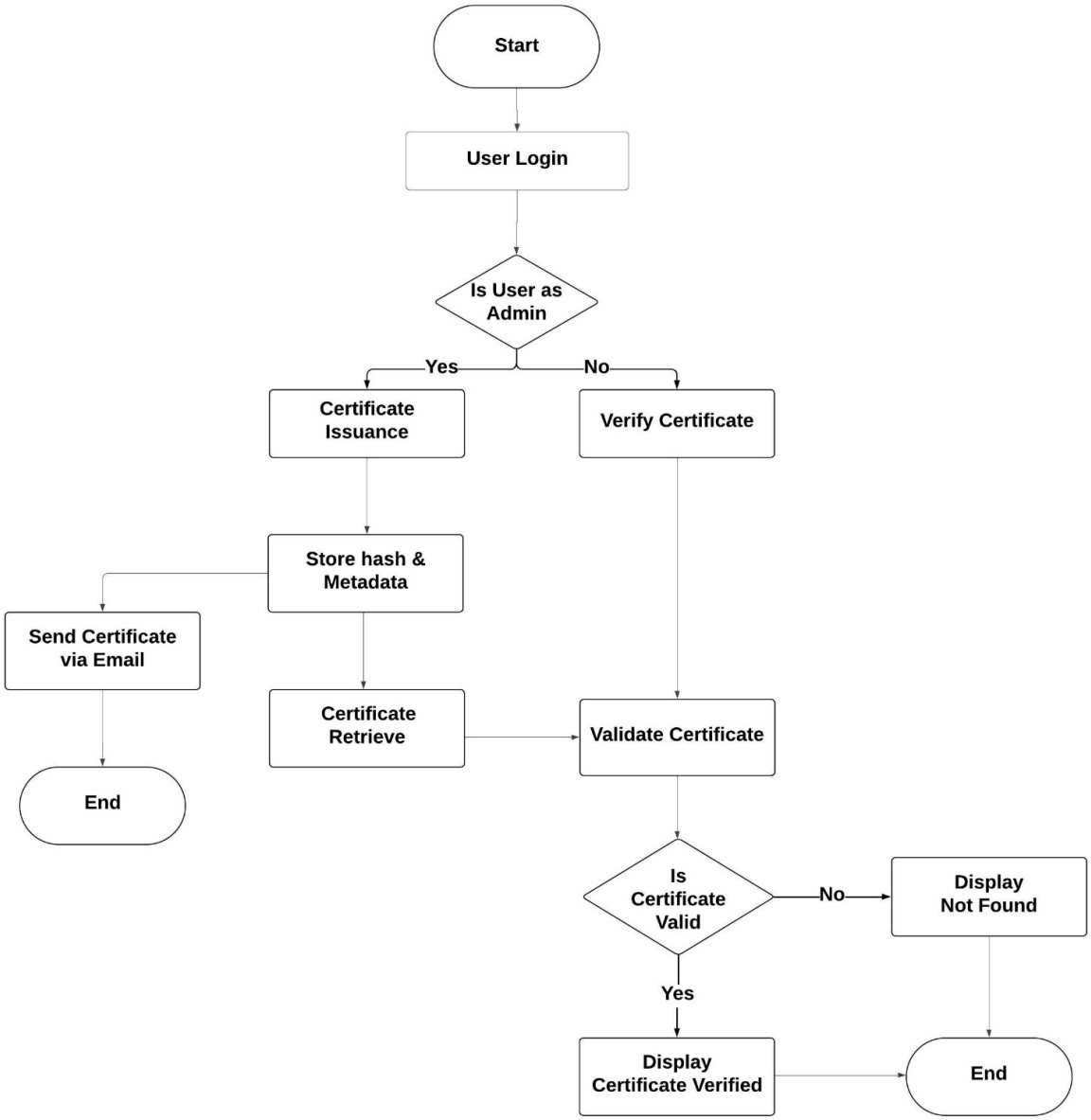


Figure 3.7 Flowchart

Figure 3.7 represents the flowchart of the academic verification, which comprises of following steps

1. Start:

The process begins with the user login step.

2. User Role Check

The system determines if the user is an Admin:

- If yes: The user can perform certificate issuance.
- If no: The user can proceed to the verification process.

3. Certificate Issuance (Admin Role):

Certificate Issuance:

- The admin enters certificate details, and the system processes the data.

Store Hash & Metadata:

- The certificate details are hashed and stored on the blockchain for security, while metadata (e.g., user ID, issuance date) is saved in the database.

4. Certificate Verification (Verifier Role):

Verify Certificate:

- The Verifier provides a Certificate ID for validation.

Retrieve Certificate:

- The system retrieves the hash from the blockchain and the associated metadata from the database.

Validate Certificate:

- The system checks whether the provided Certificate ID matches the hash stored in the blockchain.

5. Validation Outcome:

Is Certificate Valid?

- If the certificate is valid:
  - The system confirms that the “certificate is verified”.
- If the certificate is invalid or not found:
  - The system displays an error message or a “Not Found” response.

6. End:

Both the issuance and verification processes terminate after their respective outcomes are determined.

CHAPTER 4
IMPLEMENTATION

This chapter includes the software tools used, implementation details and frontend form design.

4.1 SOFTWARE TOOLS USED

The platform leverages a robust combination of tools and technologies to ensure secure, efficient, and user-friendly operations. The key software tools include:

1. **Blockchain Technology (Ethereum):** Ethereum is used to securely store certificate hashes. The Ropsten Test Network allows developers to deploy and test smart contracts without incurring real-world costs. Ensures immutability, transparency, and decentralized storage of critical data.
2. **Smart Contracts (Solidity):** Smart contracts written in Solidity manage certificate issuance, retrieval, and validation. Automates the process, ensuring secure operations without intermediaries.
3. **Backend Development (Node.js):** Node.js is used to create APIs for interactions between the front-end, blockchain, and database. Provides a scalable, event-driven architecture for handling multiple requests.
4. **Database (MongoDB):** Non-sensitive certificate metadata and user information are stored in MongoDB. Reduces blockchain storage costs and facilitates quick queries.
5. **Frontend Framework (React.js):** React.js is used to build the system's interface, ensuring a user-friendly and responsive experience. Provides an interactive platform for users to issue, view, and verify certificates.
6. **Development Environment Tools:**
   Ganache: A local blockchain emulator for testing smart contracts.
   Truffle: A development framework for writing, testing, and deploying smart contracts.
   Web3.js: Facilitates interaction between the application and Ethereum blockchain.
   Metamask: A browser extension for managing Ethereum accounts and signing transactions.
7. **Integrated Development Environment (IDE):**
   Visual Studio Code (VS Code): Visual Studio Code serves as our Integrated Development Environment (IDE) for writing, editing, and managing the codebase of

our project. Its extensive features, including syntax highlighting, code completion, debugging capabilities, and seamless integration with version control systems, enhance our productivity and facilitate collaborative development.

4.2 Implementation Details

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.19;

contract Certificate {
   mapping(string => bytes32) public certificates;
   event CertificateIssued(string certificateId, bytes32 certificateHash);

   function issueCertificate(string memory certificateId, bytes32 certificateHash) public {
      require(certificates[certificateId] == 0, "Certificate already exists");
      certificates[certificateId] = certificateHash;
      emit CertificateIssued(certificateId, certificateHash);
   }

   function verifyCertificate(string memory certificateId, bytes32 certificateHash) public view returns (bool) {
      return certificates[certificateId] == certificateHash;
   }
}
```

4.3 FRONT END FORMS DESIGN

The front-end form design section typically provides details about the user interface elements and layout designed for data input.

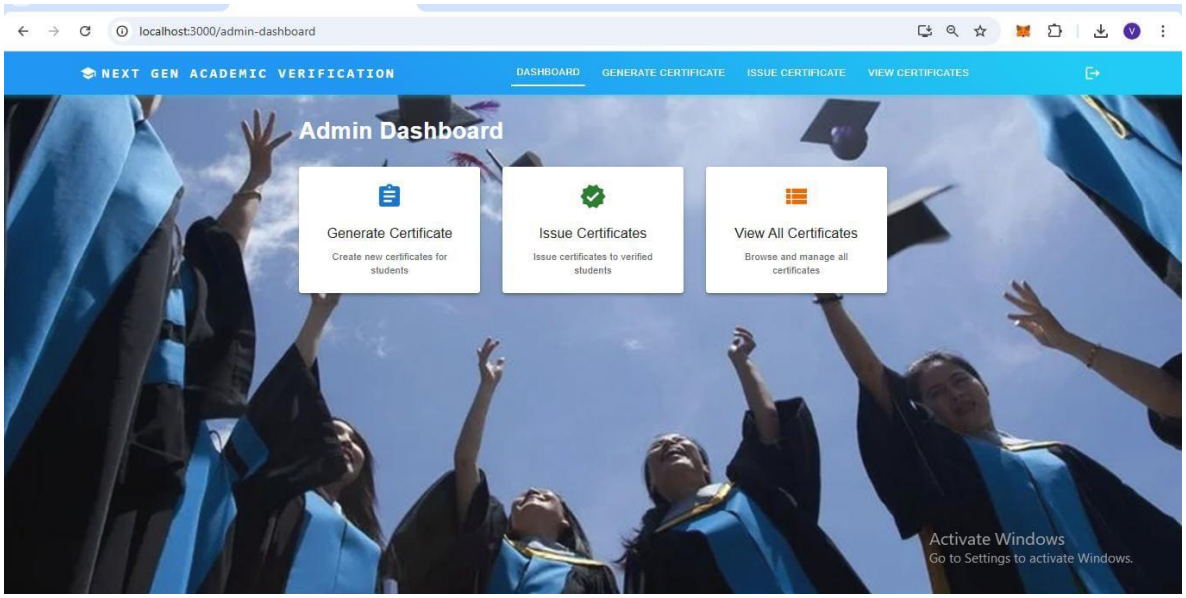


Figure 4.1 Admin Dashboard

Figure 4.1 represents the admin dashboard page where the admin can generate certificate, issue certificate and view all the certificate through the admin credentials.

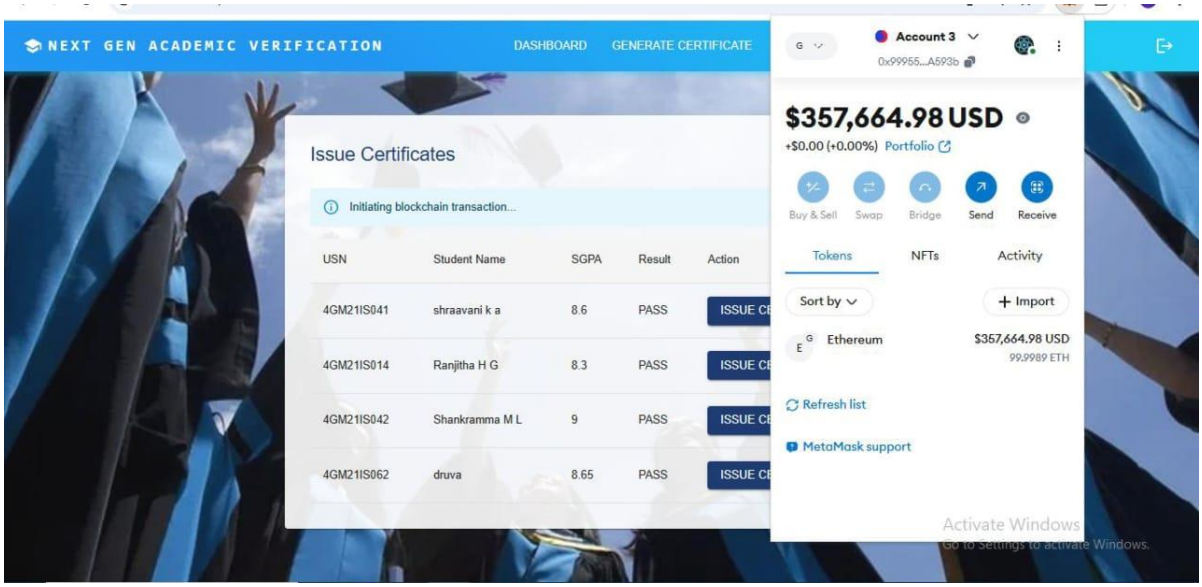


Figure 4.2 Metamask Transaction Initialization

Figure 4.2 represents the metamask transaction initialization which shows the ethers (ETH) in the admin's account and activity.

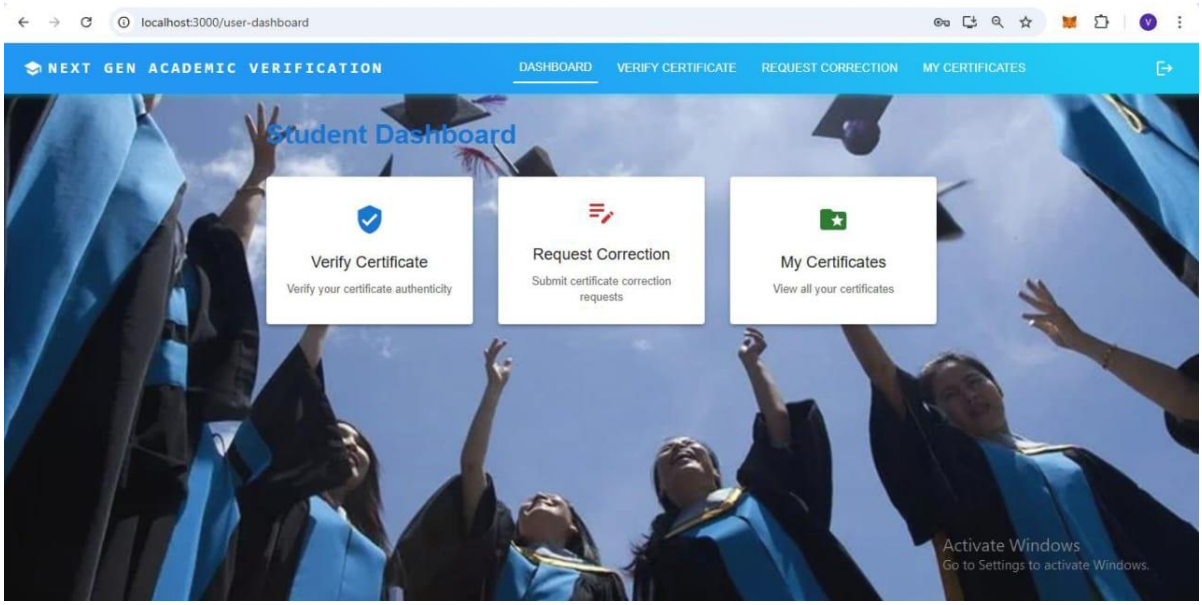


Figure 4.3 Student Dashboard

Figure 4.3 represents student dashboard presenting certificates, their verification and request for correction.

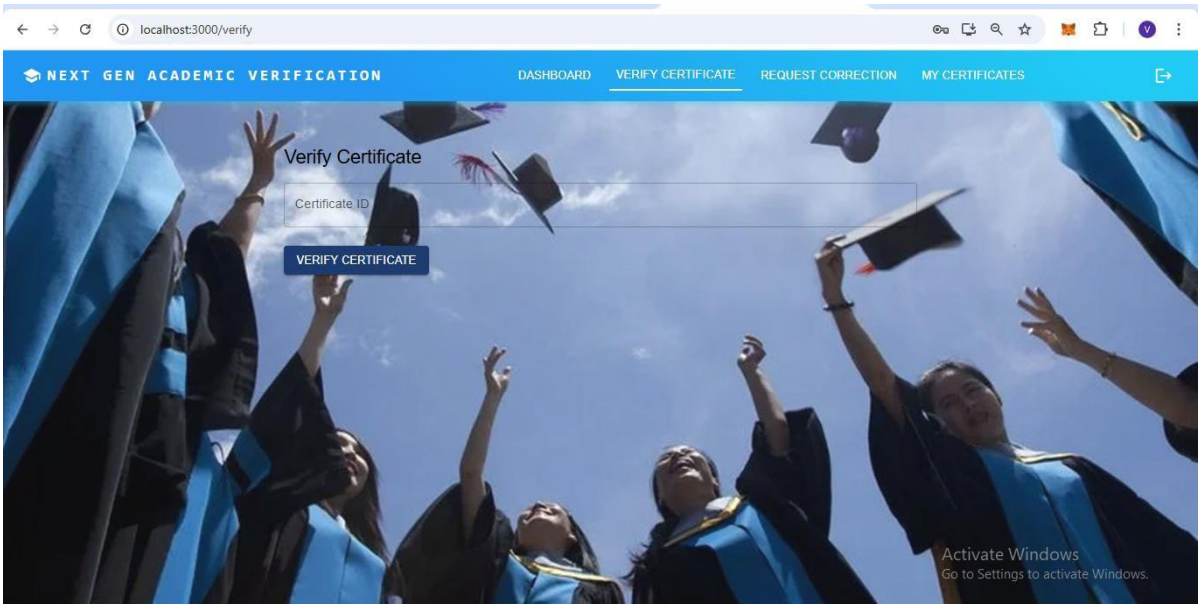


Figure 4.4 Verify Certificate

Figure 4.4 represents student's verify certificate page where students can verify their certificate by providing Certificate ID.

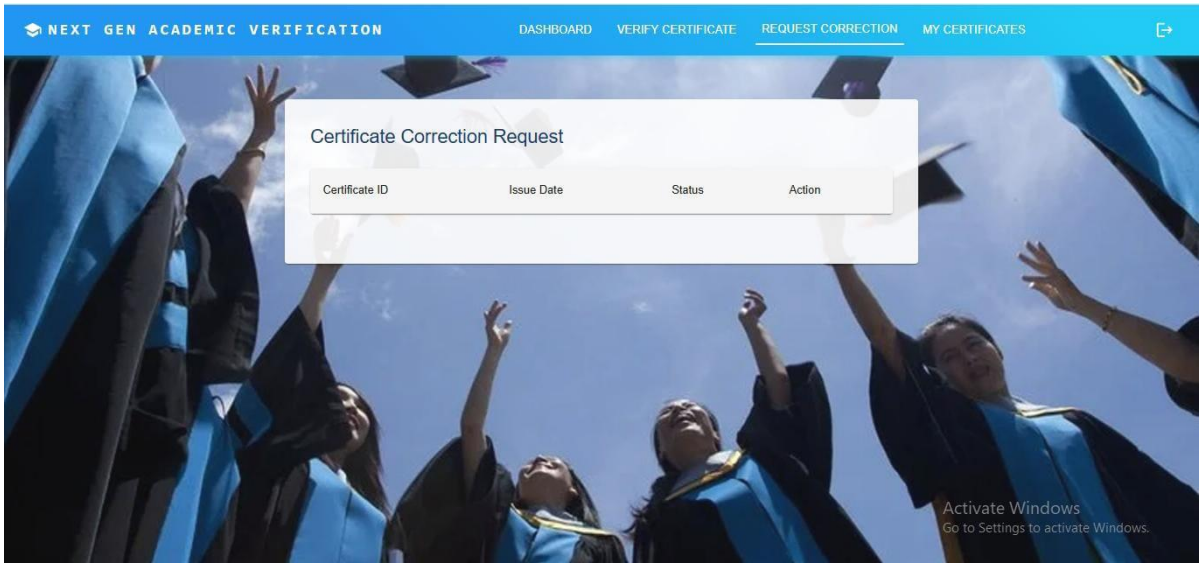


Figure 4.5 Certificate Correction Request

Figure 4.5 represents the correction request page where student can view their corrected certificates.

CHAPTER 5

TESTING

This section describes scope and testing details of our project.

5.1 SCOPE

The scope of the Blockchain-Based Certificate Authentication System encompasses the development of a web application designed to streamline the issuance and verification of educational certificates. This project will primarily focus on the following areas:

1. User Interface Development:

The project will include the creation of an intuitive and user-friendly interface for both students and administrators. This will facilitate easy navigation and interaction with the system.

2. Certificate Management:

The system will allow educational institutions to manage certificate issuance efficiently. Administrators will have access to tools for tracking and updating issued certificates, enhancing administrative processes.

3. Integration with Blockchain:

While the initial version may not fully implement blockchain technology, the scope will include planning for future integration. This will provide a decentralized and tamper-proof solution for certificate verification, which can significantly enhance data security.

4. Email Notification System:

The scope includes the implementation of an email notification system that will alert users upon successful certificate issuance, along with essential details regarding their certificates.

5. Security Measures:

The project will focus on implementing robust security measures to protect user data, including password hashing, input validation, and error handling.

6. User Support:

The project will also consider providing user support and documentation to assist users in navigating the system and resolving potential issues.

5.2 UNIT TESTING:

Unit testing involves testing individual components or units of code in isolation to ensure they function correctly.

Table 5.1 Unit Testing

Test Description	Test Input	Expected Output	Actual Output	Pass/Fail
Certificate Issuance	Certificate data	Hash stored on the blockchain	Hash stored correctly	Pass
Certificate Verification	Valid/invalid certificate hashes	Correct validation or rejection	Correct validation or rejection	Pass
API Endpoint Testing	Valid/invalid API calls	Correct responses	Correct responses	Pass
Frontend Form Submission	Valid/invalid form inputs	Valid/invalid form inputs	Behaved as expected	Pass
Database CRUD Operations	Certificate metadata	CRUD operations successful	CRUD operations successful	Pass

Table 5.1 represents the unit testing focuses on evaluating individual components of a system to ensure they function as expected. In the context of the project, it includes testing different aspects like smart contracts, backend APIs, frontend components, and database operations. For smart contracts, tests check if the certificate's hash is stored correctly, if verification logic works, and if errors are handled for invalid or duplicate entries. For APIs, tests ensure correct responses when interacting with certificate creation or verification endpoints. Frontend testing checks if user inputs trigger the correct form submissions, and database testing ensures data is stored and retrieved efficiently.

5.3 INTEGRATION TESTING

Integration testing verifies the interactions and interfaces between different components or modules of the system. It ensures that individual units work together as expected when integrated.

Table 5.2 Integration Testing

Test Description	Test Input	Expected Output	Actual Output	Pass/Fail
Frontend-Backend Interaction	User clicks "Issue Certificate" button	Blockchain transaction is triggered and confirmed	Transaction successfully triggered	Pass
Blockchain Integration	API sends a hash to Ethereum	Certificate hash is stored and retrievable	Hash stored and retrieved correctly	Pass
Database Synchronization	Database update request	MongoDB reflects accurate updates and syncs with front-end	Synchronization observed	Pass
Third-Party Services	Certificate issuance data	Email confirmation is sent to the recipient	Email sent successfully	Pass

Table 5.2 represents the integration testing focuses on verifying that different parts of the system work together as expected. It checks if the user interface, built with React, communicates smoothly with the backend APIs powered by Node.js. It also ensures that the system interacts correctly with blockchain technology, confirming that data sent to the blockchain is retrievable and intact. The test also checks the synchronization between the database and blockchain, ensuring that updates in one system are reflected accurately in the other. Lastly, it ensures proper integration with external tools like Metamask and email services to confirm that notifications are sent successfully.

5.4 FUNCTIONAL TESTING

Functional testing evaluates the functionality of the system as a whole to ensure it meets the specified requirements. It focuses on testing the system's features and capabilities from an end-user perspective.

Table 5.3 Functional Testing

Test Description	Test Input	Expected Output	Actual Output	Pass/Fail
Certificate Issuance	Admin inputs certificate data	Data stored and email sent	Successful	Pass
Certificate Verification	Verifier enters valid ID	Status displayed accurately	Accurate status shown	Pass
User Authentication	User logs in with credential	Dashboard displayed by user role	Correct dashboard shown	Pass
Error Scenario Invalid ID	Verifier enters invalid ID	Certificate not found error shown	Error displayed	Pass

Table 5.3 below represents the functional testing for our website: In our project, functional testing would involve testing the core functionalities, including certificate issuance, verification, user authentication, and error handling. The focus would be on ensuring that all system processes work as expected when interacting with users.

5.5 SYSTEM TESTING

System testing involves evaluating the entire software system as a whole to ensure that it meets the specified requirements and functions correctly in its intended environment. This comprehensive testing phase verifies the system's functionality, performance, reliability, security, and usability.

Table 5.3 System testing

Test Description	Test Input	Expected Output	Actual Output	Pass/Fail
End-to-End Workflows	Issue, verify certificate, handle errors	Seamless experience, accurate data	Workflow executed successfully	Pass
Load Testing	High traffic with multiple verifiers	Acceptable response times	System responded efficiently	Pass
Security Testing	Blockchain and MongoDB checks	Secure transactions, no issues	Secure transactions validated	Pass
Cross-Browser Compatibility	Test on Chrome, Firefox, Safari	Consistent behaviour	Consistency observed	Pass

The table 5.3 outlines the system testing process for the academic certificate verification system. It includes different test types such as end-to-end workflows, load testing, security testing, and cross-browser compatibility. Each test describes the input scenario, expected output, and the actual output observed during testing. The "Pass/Fail" column indicates whether the system met the expected results. The testing ensures the system functions smoothly, handles high traffic, maintains secure transactions, and works consistently across multiple browsers.

CHAPTER 6

SNAPSHOTS OF THE PROJECT

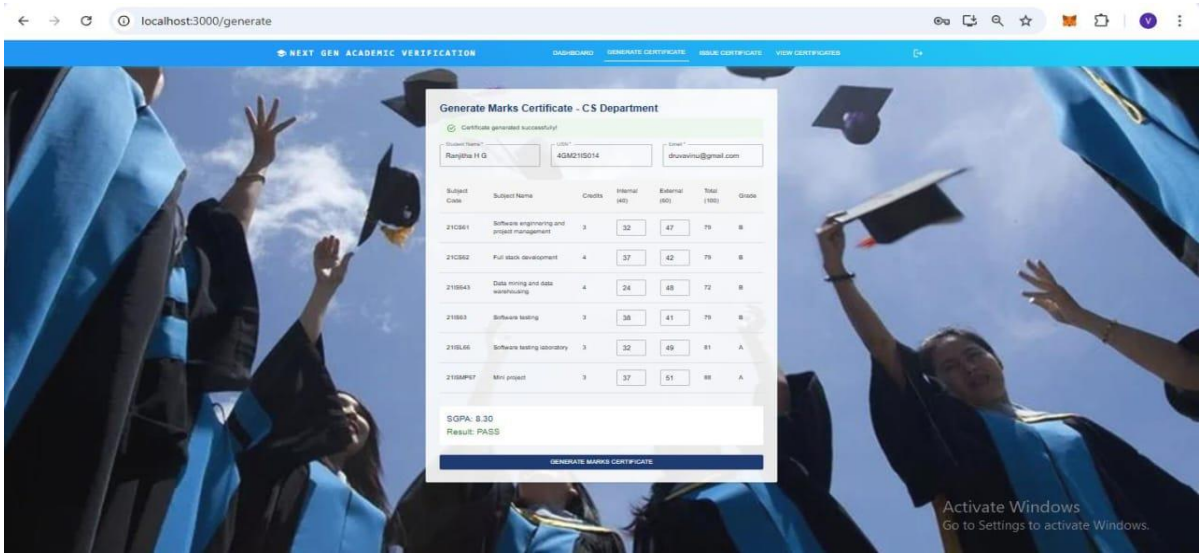


Figure 6.1 Generate Marks Certificate

Figure 6.1 shows the generate certificate page where admin uploads student details, subject code, name, credits and marks. SGPA and result are calculated based on this given data.

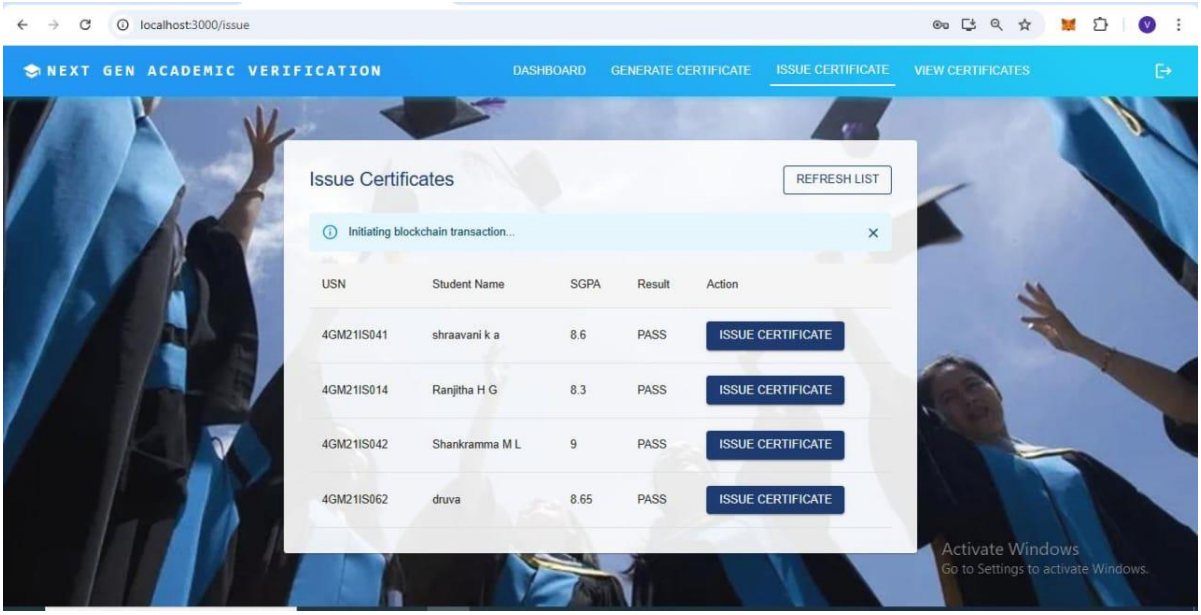


Figure 6.2 Issue Certificate

Figure 6.2 shows the issue certificate page where all the generated certificates are shown here. The admin issues the certificate by initiating blockchain transaction.

CHAPTER 7

CONCLUSION

7.1 CONCLUSION

The Blockchain-Based Certificate Authentication System addresses the prevalent issues of fraud, inefficiency, and lack of trust in traditional certificate management systems. By leveraging the transparency, security, and immutability of blockchain technology, this project provides a robust platform for educational institutions and employers.

Certificates are issued, stored, and verified securely, ensuring authenticity and eliminating forgery. The integration of Ethereum's blockchain with MongoDB for off-chain storage creates a scalable and efficient solution. The system's features, such as tamper-proof data, streamlined verification, and user notifications, make it highly effective for real-world applications. Testing the system in various scenarios demonstrated its reliability, scalability, and ease of use. The project meets its objectives, showcasing the potential of blockchain technology in improving societal trust in certificate management.

This project is a significant step toward digitizing and securing academic and professional credentials, marking a positive impact on the education and employment sectors.

7.2 FUTURE SCOPE

The system has the potential for significant advancements to enhance functionality, scalability, and user experience. Future developments may include:

1. **Integration with National Databases:** Linking with centralized educational and professional databases for wider adoption and scalability.
2. **Multi-Chain Support:** Expanding to support multiple blockchain networks to ensure resilience and cost optimization.
3. **AI-Based Fraud Detection:** Incorporating AI to detect anomalies in certificate issuance or verification processes.
4. **Enhanced User Features:** Enabling students to request certificate edits or duplicate copies through the system.
5. **Interoperability:** Building APIs for seamless integration with HR systems, government portals, and academic platforms.

APPENDIX-A

1. Ethereum

Ethereum is a decentralized platform designed to facilitate the creation and execution of smart contracts and decentralized applications (DApps). Launched in 2015 by Vitalik Buterin, Ethereum operates on blockchain technology, which ensures transparency, immutability, and security. Unlike Bitcoin, which focuses primarily on cryptocurrency transactions, Ethereum is built to support programmable contracts and applications via its native programming language, Solidity.

At the core of Ethereum is the Ethereum Virtual Machine (EVM), a decentralized computation engine that allows developers to execute smart contracts across the network. This functionality makes Ethereum a powerful tool for creating blockchain solutions for a wide variety of industries, such as finance, healthcare, and education. Each transaction or operation on the Ethereum network requires "gas," a fee paid in Ether (ETH), Ethereum's native cryptocurrency. This incentivizes miners or validators to maintain the network.

Ethereum operates on a decentralized peer-to-peer network, meaning there is no central authority overseeing the operations. This decentralization ensures that applications built on Ethereum are resilient against censorship and downtime. The introduction of Ethereum 2.0 brought significant improvements in scalability, security, and energy efficiency by transitioning from a Proof-of-Work (PoW) to a Proof-of-Stake (PoS) consensus mechanism.

2. Smart Contracts

Smart contracts are self-executing digital agreements embedded with predefined rules and conditions. These contracts automatically enforce the agreed-upon terms once the conditions are met, eliminating the need for intermediaries. Hosted on the blockchain, smart contracts ensure transparency and immutability, making them highly reliable for sensitive operations.

Developed using programming languages such as Solidity, smart contracts operate within a decentralized framework. Once deployed, they become tamper-proof, as any alterations require consensus from the network participants. Smart contracts are widely used for applications like finance, supply chain management, and academic verification. For example, in blockchain-based certificate verification systems, a smart contract can hash and store certificate details on the blockchain, allowing for quick, automated validation. Their efficiency and trustless execution make smart contracts a transformative tool in various domains.

REFERENCES

[1]. Nakamoto, S. (2008) Bitcoin: A Peer-to-Peer Electronic Cash System. https://www.bitcoin.org

[2]. Zheng, Z., Xie, S., Dai, H., Chen, X. and Wang, H. (2017) An Overview of Blockchain Technology: Architecture, Consensus, and Future Trends. Proceedings of 2017 IEEE 6th International Congress on Big Data (BigData Congress), Honolulu, 25-30 June 2017, 557-564. https://doi.org/10.1109/BigDataCongress.2017.85

[3]. Garg, R. (2021) Blockchain Ecosystem for Education & Employment Verification. Proceedings of 13th International Conference on Network & Communication Secu- rity (NCS 2021), Toronto, 25-26 September 2021.

[4]. Ghazali, O. and Saleh, O.S. (2018) A Graduation Certificate Verification Model via Utilization of the Blockchain Technology. JTEC: Journal of Telecommunication, Electronic and Computer Engineering, 10, 29-34.

[5]. Dongre, J.G., Tikam, S.M. and Gharat, V.B. (2020) Education Degree Fraud Detec- tion and Student Certificate Verification Using Blockchain. IJERT, 9, 300-303. https://www.ijert.org

[6]. Suganthalakshmi, M.R., Praba, M.G.C., Abhirami, M.K. and Puvaneswari, M.S. (2022) Blockchain Based Certificate Validation System. https://www.irjmets.com